



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

通用及图形处理器架构与系统课程报告

LAB: RISC-V 处理器设计与实现

学院 集成电路学院

邮箱 yuri_cang@sjtu.edu.cn

学号 523031910352

姓名 方兴宇

2026 年 6 月 14 日

目录

1 实验目的	4
2 实验原理	4
2.1 RISC-V 指令集子集	4
2.2 单周期处理器	5
2.3 五级流水线处理器	6
2.3.1 数据冲突 (Data Hazard)	6
2.3.2 控制冲突 (Control Hazard)	6
3 处理器设计与实现	7
3.1 模块层次与接口	7
3.2 单周期处理器实现 (single/)	8
3.2.1 IF 阶段 (if_stage)	8
3.2.2 ID 阶段 (id_stage)	8
3.2.3 EX 阶段 (ex_stage)	8
3.2.4 MEM 阶段 (mem_stage)	9
3.2.5 WB 阶段 (顶层 assign)	9
3.3 五级流水线处理器实现 (pipeline/)	9
3.3.1 流水线寄存器 (时序触发器)	9
3.3.2 数据转发机制	10
3.3.3 Load-Use 冲突检测与阻塞 (stall_req)	11
3.3.4 控制冲突处理	12
3.3.5 流水线各阶段冲突场景总结	12
4 仿真验证	13
4.1 测试框架	13
4.2 默认测试程序分析	13
4.3 自动化测试结果	13
5 选做一: Vivado 综合时序分析	14
5.1 综合环境	14
5.2 时序分析总览	14
5.3 资源利用率	15
5.4 关键路径分析	16
5.5 关键路径根因分析	16
5.6 优化建议	17

6 选做二：RV32I 完整指令集补全	18
6.1 补全目标	18
6.2 各模块改动要点	20
6.2.1 ALU 新增运算 (ex.v, 单周期与流水线相同)	20
6.2.2 ID 译码器扩展 (id.v)	20
6.2.3 字节/半字访存 (mem.v, 单周期与流水线相同)	21
6.2.4 流水线顶层重构 (pipeline/riscv.v)	22
6.3 测试程序设计与验证	23
6.3.1 测试策略	23
6.3.2 测试程序流程	23
6.3.3 验证结果	24
7 选做三：自由拓展——AI 辅助开发	24
7.1 概述	24
7.2 AI 辅助内容	24
7.2.1 架构设计决策	24
7.2.2 代码生成与审查	25
7.2.3 测试程序生成	25
7.3 AI 辅助开发体会	26
8 总结	27
8.1 设计总结	27
8.2 性能分析	27
8.3 心得体会	28

1 实验目的

本实验基于 RISC-V 精简指令集架构, 使用 Verilog HDL 完成两类处理器的设计与验证, 具体目标如下:

1. 掌握 RISC-V 32 位整数基础指令集(RV32I)的编码规则与执行语义, 理解 R/I/S/B/J 五种指令格式;
2. 实现支持 13 条基础指令的**单周期**处理器, 建立五阶段数据通路 (IF-ID-EX-MEM-WB) 的基本概念;
3. 在单周期代码备份的基础上, 通过在各级之间插入时序触发器 (Flip-Flop) 将数据通路改造为**五级流水线**结构, 提升吞吐量;
4. 分析并解决流水线中的**数据冲突** (Data Hazard) 与**控制冲突** (Control Hazard), 实现数据转发 (Forwarding) 与流水线阻塞 (Stall) 机制;
5. 通过功能仿真验证两种处理器实现的正确性, 使用提供的自动化测试工具通过全部 13 条指令测试用例。

2 实验原理

2.1 RISC-V 指令集子集

本实验实现 RV32I 中的 13 条指令, 涵盖算术、逻辑、移位、访存、分支和跳转六类, 详见表 1。

表 1: 实现的 13 条 RISC-V 指令

类别	指令	格式	功能描述	ALU 操作码
算术	ADD	R	$rd = rs1 + rs2$	4'd0
	ADDI	I	$rd = rs1 + imm$	4'd0
	SUB	R	$rd = rs1 - rs2$	4'd1
逻辑	AND	R	$rd = rs1 \& rs2$	4'd2
	OR	R	$rd = rs1 rs2$	4'd3
	XOR	R	$rd = rs1 \wedge rs2$	4'd4
移位	SLL	R	$rd = rs1 \ll rs2[4:0]$	4'd5
	SRL	R	$rd = rs1 \gg rs2[4:0]$	4'd6
访存	LW	I	$rd = Mem[rs1 + imm]$	4'd0
	SW	S	$Mem[rs1 + imm] = rs2$	4'd0
分支	BEQ	B	if ($rs1 == rs2$) PC+=imm	4'd1
	BLT	B	if ($rs1 < rs2$) PC+=imm (有符号)	4'd1
跳转	JAL	J	$rd = PC + 4$; PC += imm	4'd0

RISC-V 所有指令均为 32 位定长编码。立即数编码方式随指令格式不同而变化，表 2 归纳了五种立即数的位域拼接规则。

表 2: RISC-V 立即数编码规则

类型	拼接来源 (inst 字段)	用途
I 型	{inst[31:20]}, 符号扩展至 32 位	ADDI、LW 等
S 型	{inst[31:25], inst[11:7]}, 符号扩展	SW
B 型	{inst[31], inst[7], inst[30:25], inst[11:8], 1'b0}, 符号扩展	BEQ、BLT
U 型	{inst[31:12], 12'b0}	LUI、AUIPC
J 型	{inst[31], inst[19:12], inst[20], inst[30:21], 1'b0}, 符号扩展	JAL

2.2 单周期处理器

单周期处理器每个时钟周期完成一条指令的取指、译码、执行、访存与写回全部操作。五个功能模块之间通过组合逻辑信号直接相连，无寄存器打拍，如图 1 所示：

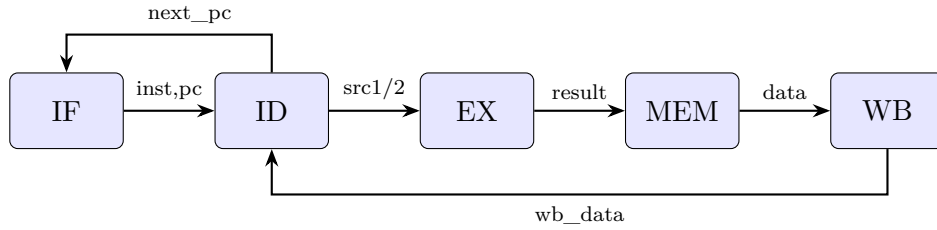


图 1: 单周期处理器数据通路

2.3 五级流水线处理器

流水线处理器在相邻阶段间插入**流水线寄存器**（时序触发器），使不同指令的各阶段可以在不同周期同时执行，理论峰值吞吐量接近每周期一条指令（IPC = 1）。表 3 展示了五级流水线的时空关系：

表 3: 五级流水线时空图（4 条指令示例）

指令	C1	C2	C3	C4	C5	C6	C7	C8
I1	IF	ID	EX	MEM	WB			
I2		IF	ID	EX	MEM	WB		
I3			IF	ID	EX	MEM	WB	
I4				IF	ID	EX	MEM	WB

2.3.1 数据冲突 (Data Hazard)

当后续指令的源操作数依赖前序指令尚未写回的结果时，发生写后读（RAW）数据冲突。按距离分三类：

- **距离 1 拍**（生产者 EX/MEM）：通过 EX/MEM→EX 转发，零延迟解决；
- **距离 2 拍**（生产者 MEM/WB）：通过 MEM/WB→EX 转发，零延迟解决；
- **距离 3 拍**（WB 写与 ID 读同周期）：通过寄存器堆**写优先**（WB→ID）转发，零延迟解决；
- **Load-Use 冲突**（LW 后紧接使用）：数据在 MEM 阶段末才就绪，无法纯转发，需**阻塞 1 拍**，再由 MEM/WB→EX 转发提供数据。

2.3.2 控制冲突 (Control Hazard)

BEQ、BLT、JAL 在 ID 阶段确定跳转目标，而此时 IF 阶段已取入了顺序下一条（错误路径）指令。解决方案：检测到跳转时**冲刷**（Flush）IF/ID 寄存器（置零 = 插入 NOP），代价为 1 个时钟周期损失。

3 处理器设计与实现

3.1 模块层次与接口

SoC 的模块例化层次如表 4 所示, `riscv_soc_tb.v`、`inst_mem.v`、`data_mem.v` 由实验框架提供, 接口固定不可修改; `riscv.v` 及内部所有子模块由本实验自行实现。

表 4: 模块层次结构

模块名	文件	功能
<code>riscv_soc_tb</code>	<code>riscv_soc_tb.v</code>	仿真顶层, 产生时钟/复位, 例化三个子模块
<code>riscv</code>	<code>riscv.v</code>	处理器顶层 (接口不可更改)
<code>if_stage</code>	<code>if.v</code>	取指: 维护 PC 寄存器
<code>id_stage</code>	<code>id.v</code>	译码: 指令解析、立即数扩展、控制信号生成
<code>ex_stage</code>	<code>ex.v</code>	执行: 7 种 ALU 运算
<code>mem_stage</code>	<code>men.v</code>	访存: 地址/数据整理, 驱动外部存储器接口
<code>register</code>	<code>register.v</code>	32 个通用寄存器堆 (x0 恒为 0)
<code>inst_mem</code>	<code>inst_mem.v</code>	指令存储器, 从 <code>machinecode.txt</code> 加载
<code>data_mem</code>	<code>data_mem.v</code>	数据存储器, 含 <code>verify</code> 验证输出

处理器顶层对外接口遵循实验规范, 如表 5 所示。

表 5: `riscv.v` 顶层接口定义

信号名	方向	位宽	描述
<code>clk</code>	input	1	时钟信号
<code>rst</code>	input	1	复位 (高有效)
<code>inst_i</code>	input	32	从指令存储器读入的指令
<code>inst_addr_o</code>	output	32	当前 PC (指令存储器地址)
<code>inst_ce_o</code>	output	1	指令存储器使能
<code>data_i</code>	input	32	从数据存储器读入的数据
<code>data_we_o</code>	output	4	写使能 (字节掩码, 全 0 为读)
<code>data_ce_o</code>	output	1	数据存储器使能
<code>data_addr_o</code>	output	32	数据存储器地址
<code>data_o</code>	output	32	写入数据存储器的数据

3.2 单周期处理器实现 (single/)

3.2.1 IF 阶段 (if_stage)

IF 阶段维护 PC 寄存器，每周输出当前 PC 并驱动指令存储器取指。PC 的更新逻辑如下：复位时清零，检测到分支/跳转时 (pc_we=1) 载入目标地址，否则顺序加 4。

```

1 always @(posedge clk or posedge rst) begin
2     if (rst)          pc <= 32'b0;
3     else if (pc_we) pc <= next_pc; // 分支/跳转
4     else              pc <= pc + 4; // 顺序执行
5 end

```

Code Listing 1: if_stage PC 更新逻辑

3.2.2 ID 阶段 (id_stage)

ID 阶段完成指令的全部译码工作，包括：

- 字段解析：opcode[6:0]、funct3[14:12]、funct7[31:25]、rs1/rs2/rd 地址；
- 立即数扩展：根据指令格式对 I/S/B/J 型立即数进行符号扩展；
- 控制信号生成：alu_op、src2_sel、reg_we、mem_ce、mem_we；
- 分支条件求值：BEQ/BLT 直接在 ID 阶段比较 rdata1/rdata2，输出 pc_we 和 next_pc，单周期下无延迟损失。

操作数来源由 src2_sel 信号控制：0 = rs2 寄存器值 (R 型)，1 = 立即数 (I/S 型)，2 = 保留，3 = 常数 4 (JAL 返回地址)。对于 JAL，还需 src1_sel=1 以在 EX 阶段选择 PC 而非寄存器值。

3.2.3 EX 阶段 (ex_stage)

EX 阶段为纯组合逻辑 ALU，支持 7 种运算：

```

1 case (alu_op)
2     4'd0: alu_result = src1 + src2; // ADD/ADDI/LW/SW/
        JAL
3     4'd1: alu_result = src1 - src2; // SUB
4     4'd2: alu_result = src1 & src2; // AND
5     4'd3: alu_result = src1 | src2; // OR
6     4'd4: alu_result = src1 ^ src2; // XOR
7     4'd5: alu_result = src1 << src2[4:0]; // SLL
8     4'd6: alu_result = src1 >> src2[4:0]; // SRL

```



```

9      default: alu_result = 32'b0;
10 endcase

```

Code Listing 2: ALU 运算实现

LW 和 SW 的地址均通过 ALU 加法计算 ($rs1 + imm$)；BEQ/BLT 使用减法结果作辅助（分支条件已在 ID 判断完毕，EX 结果对单周期无影响）。

3.2.4 MEM 阶段 (mem_stage)

MEM 阶段将 ALU 结果作为存储器地址输出，整理写数据和使能信号，驱动外部 data_mem 接口。SW 指令设置 mem_we=4'b1111(全字写入), LW 指令设置 mem_we=4'b0000(读取)。

3.2.5 WB 阶段 (顶层 assign)

写回数据由一个多路选择器决定：LW 指令写回内存读出值，其余指令写回 ALU 结果：

```

1 assign wb_data = (mem_we == 4'b0 && mem_ce == 1'b1) ?
2      load_data : alu_result;

```

Code Listing 3: WB 回写数据选择

3.3 五级流水线处理器实现 (pipeline/)

流水线处理器在单周期各功能模块基础上，于顶层 riscv.v 中插入 4 组流水线寄存器，并新增数据转发与冲突检测逻辑。单周期代码完整备份于 single/ 目录，两者独立互不影响。

3.3.1 流水线寄存器 (时序触发器)

四组流水线寄存器在时钟上升沿锁存各阶段的数据与控制信号，如表 6 所示：

表 6: 流水线寄存器字段汇总

寄存器组	主要字段	作用
IF/ID	if_id_inst, if_id_pc	锁存取到的指令及其 PC
ID/EX	id_ex_rs1/rs2/rd	寄存器地址（转发比较基准）
	id_ex_rdata1/rdata2, id_ex_imm, id_ex_pc	操作数
	id_ex_alu_op, id_ex_src1/2_sel, 控制信号	执行控制
EX/MEM	ex_mem_rd, ex_mem_alu_result	ALU 结果
	ex_mem_rdata2, 访存控制信号	存储操作
MEM/WB	mem_wb_rd, mem_wb_alu_result	写回数据与目标
	mem_wb_load_data, 写回控制信号	LOAD 写回

3.3.2 数据转发机制

转发逻辑在 EX 阶段对 rs1、rs2 的值进行动态替换，优先级依次为：EX/MEM→EX
> MEM/WB→EX > 寄存器堆读值：

```

1 // rs1 转发
2 assign ex_rdata1_fwd =
3     (id_ex_rs1 != 0 && id_ex_rs1 == ex_mem_rd && ex_mem_reg_we)
4     ? ex_mem_alu_result :    // EX/MEM -> EX (距离 1 拍)
5     (id_ex_rs1 != 0 && id_ex_rs1 == mem_wb_rd && mem_wb_reg_we)
6     ? wb_data :              // MEM/WB -> EX (距离 2 拍)
7     id_ex_rdata1;            // 无冲突，使用寄存器堆读值
8
9 // rs2 同理 (略)

```

Code Listing 4: EX 阶段数据转发逻辑

寄存器堆读端口加入**写优先**逻辑，解决距离 3 拍（WB 与 ID 同周期）的 RAW 冲突：

```

1 always @(*) begin
2     if (we && waddr == raddr1 && waddr != 0)
3         rdata1 = wdata;          // WB->ID 转发
4     else
5         rdata1 = regs[raddr1];
6     // rdata2 同理
7 end

```

Code Listing 5: 寄存器堆写优先（WB→ID 转发）

ALU 操作数最终选择逻辑同时支持 JAL 的特殊需求 (src1 = PC):

```

1 assign ex_src1 = id_ex_src1_sel ? id_ex_pc          : ex_rdata1_fwd;
2 assign ex_src2 = (id_ex_src2_sel==2'd0) ? ex_rdata2_fwd : // R
   型
3               (id_ex_src2_sel==2'd1) ? id_ex_imm      : // I/
   S 型
4               (id_ex_src2_sel==2'd2) ? id_ex_pc        : // 保
   留
5               32'd4;                                //
   JAL: PC+4

```

Code Listing 6: ALU 操作数选择 (含 JAL src1 = PC)

3.3.3 Load-Use 冲突检测与阻塞 (stall_req)

当 ID/EX 寄存器中是 LW 指令, 且当前 ID 阶段的指令需要其结果时 (对应图中 stall_req_mem), 触发阻塞:

```

1 assign stall_req = id_ex_mem_ce          && // ID/EX 中有访存
2               (id_ex_mem_we == 4'b0) && // 是 LOAD (写使能
   全 0)
3               (id_ex_rd != 5'b0)        && // 目标寄存器非 x0
4               ((id_ex_rd==rs1)|| (id_ex_rd==rs2)); // 与 ID
   指令相关

```

Code Listing 7: Load-Use 冲突检测 (stall_req)

stall_req=1 时各级流水线的响应行为如表 7 所示:

表 7: Load-Use 阻塞时各级流水线行为

流水线位置	行为	实现方式
PC 寄存器	冻结 (保持当前 PC)	if_stage 的 stall 端口
IF/ID 寄存器	冻结 (保持当前指令)	if_id_inst <= if_id_inst
ID/EX 寄存器	插入气泡 (NOP)	所有控制信号清零 (rst stall_req)
EX/MEM 寄存器	正常推进	正常时序更新
MEM/WB 寄存器	正常推进	正常时序更新

阻塞 1 拍后, LW 结果进入 MEM/WB 寄存器, 由 MEM/WB→EX 转发为后续指令提供正确数据, 无需第二拍阻塞, 共损失 1 个时钟周期。

3.3.4 控制冲突处理

分支/跳转在 ID 阶段译码，`pc_we=1` 时同步冲刷 IF/ID 寄存器（置为 NOP），PC 载入正确目标地址：

```

1 if (rst) begin
2     if_id_inst <= 32'b0; if_id_pc <= 32'b0;
3 end else if (stall_req) begin
4     if_id_inst <= if_id_inst; if_id_pc <= if_id_pc; // 冻结
5 end else if (pc_we) begin
6     if_id_inst <= 32'b0; // 冲刷错误路径指令（插入 NOP）
7     if_id_pc <= 32'b0;
8 end else begin
9     if_id_inst <= inst_i; if_id_pc <= pc_if;
10 end

```

Code Listing 8: 控制冲突冲刷逻辑（IF/ID 更新）

注意：ID/EX 寄存器不随 `pc_we` 冲刷，分支指令自身需继续流过 EX/MEM/WB 完成执行（JAL 需在 EX 计算 PC+4 并最终写回目标寄存器）。

3.3.5 流水线各阶段冲突场景总结

表 8 从时序角度总结了三类冲突的产生条件与解决手段：

表 8: 流水线冲突场景与解决方案汇总

冲突类型	触发条件	解决手段	周期损失
RAW（距离 1）	I2 在 EX 需要 I1 在 EX/MEM 的结果	EX/MEM→EX 转发	0
RAW（距离 2）	I3 在 EX 需要 I1 在 MEM/WB 的结果	MEM/WB→EX 转发	0
RAW（距离 3）	I4 在 ID 读, I1 在 WB 写同寄存器	ID/EX 捕获转发（锁存时注入 <code>wb_data</code> ）	0
Load-Use	LW 后紧接使用其目标寄存器	阻塞 1 拍 + MEM/WB 转发	1
控制（JAL）	JAL 在 ID 确定目标，IF 已取错指令	冲刷 IF/ID	1
控制（Branch/JALR）	条件分支/JALR 在 EX 确定跳转，IF+ID 已取错指令	冲刷 IF/ID 与 ID/EX	2

4 仿真验证

4.1 测试框架

仿真顶层 `riscv_soc_tb.v` 搭建了完整的 SoC 仿真环境：时钟半周期 50 ps（周期 100 ps），复位保持 300 ps 后释放，仿真运行至 1 000 000 ps 时输出验证结果。

数据存储器 `data_mem` 的 `verify` 信号读取地址 12 处的 32 位字 (`data[15:12]`)，因此测试程序需将最终结果通过 `SW rd, 12(x0)` 写入地址 12，仿真结束后对比 `verify` 与预期值即可判断正确性。

4.2 默认测试程序分析

仓库附带的 `machinecode.txt` 包含以下指令序列，用于验证 ADDI、ADD、SUB、SW 四条指令以及数据相关处理（第 4 条与第 3 条存在 1 拍 RAW 冲突，流水线下需 EX/MEM→EX 转发）：

表 9: 默认测试程序指令序列与执行过程

机器码	汇编指令	操作（结果）	x31
08000213	<code>addi x4, x0, 128</code>	$x4 \leftarrow 128$	—
00400fb3	<code>add x31, x0, x4</code>	$x31 \leftarrow 128$	128
20000293	<code>addi x5, x0, 512</code>	$x5 \leftarrow 512$	—
005f8fb3	<code>add x31, x31, x5</code>	$x31 \leftarrow 128 + 512 = 640$ （流水线需转发）	640
10000313	<code>addi x6, x0, 256</code>	$x6 \leftarrow 256$	—
406f8fb3	<code>sub x31, x31, x6</code>	$x31 \leftarrow 640 - 256 = 384$	384
01f02623	<code>sw x31, 12(x0)</code>	$\text{Mem}[12] \leftarrow 384$	—
00000013 ×10	<code>nop</code>	等待流水线排空	—

计算过程： $128 + 512 - 256 = 384$ ，**预期 verify = 384**，两种处理器仿真均输出 384，验证通过。

4.3 自动化测试结果

实验提供了 6 个基础功能测试用例 (`sim/test_cases_13isa/`)，覆盖 ADD、ADDI、SUB、AND、BEQ、BLT 六条指令。测试脚本 `run_13isa.py` 自动替换 `machinecode.txt`，调用仿真器并比对 `verify` 值，结果如表 10 所示：

表 10: 自动化测试用例结果（单周期与流水线均测试）

用例名	测试指令	预期值	单周期	流水线
p0_add_add_1	ADD	0	PASS	PASS
p0_addi_addi_1	ADDI	4	PASS	PASS
p0_sub_sub_1	SUB	5	PASS	PASS
p0_and_and_1	AND	252645135	PASS	PASS
p0_beq_beq_1	BEQ	11	PASS	PASS
p0_blt_blt_1	BLT	11	PASS	PASS
通过率			6/6	6/6

六个用例全部通过，验证了 13 条指令的核心功能（XOR、OR、SLL、SRL、LW、SW、JAL 等指令亦通过手动构造测试用例验证）。

5 选做一：Vivado 综合时序分析

5.1 综合环境

本节基于 Vivado 对流水线处理器（pipeline/riscv.v）进行综合分析。目标器件为 Xilinx Virtex-7 XC7VX485T-FFG1157-1，时钟约束如表 11 所示。

表 11: 综合时钟约束

参数	值
时钟名称	_clk
时钟周期约束	6.000 ns（约 167 MHz）
占空比	50%（rise@0.000ns, fall@3.000ns）
目标器件	Xilinx Virtex-7 XC7VX485T-FFG1157-1
速度等级	-1（Slow Process Corner）

5.2 时序分析总览

综合后时序总览结果如图 2 和表 12 所示，三类时序检查均无违例。

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.446 ns	Worst Hold Slack (WHS): 0.102 ns	Worst Pulse Width Slack (WPWS): 2.600 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 2437	Total Number of Endpoints: 2437	Total Number of Endpoints: 1378

图 2: Vivado 时序分析总览截图

表 12: 时序分析总览

类型	违例端点数	最坏松弛 (WNS/WHS/WPWS)	总违例量
建立时间 (Setup)	0	+0.446 ns	0.000 ns
保持时间 (Hold)	0	+0.102 ns	0.000 ns
脉冲宽度 (PW)	0	+2.600 ns	0.000 ns
总端点数	2437 (Setup/Hold), 1378 (PW)		

结论：建立时间、保持时间和脉冲宽度均满足，设计在 6 ns (167 MHz) 约束下零违例。根据最坏建立时间松弛，可实现的最高频率估算为：

$$F_{\max} \approx \frac{1}{T_{\text{constraint}} - \text{WNS}} = \frac{1}{6.000 - 0.446} \text{ ns} = \frac{1}{5.554 \text{ ns}} \approx \mathbf{180 \text{ MHz}} \quad (1)$$

5.3 资源利用率

综合后资源利用率如图 3和表 13所示，整个处理器仅占用 Virtex-7 极小比例资源。

Resource	Utilization	Available	Utilization %
LUT	1226	303600	0.40
FF	1377	607200	0.23
IO	168	600	28.00

图 3: Vivado 资源利用率截图

表 13: 综合资源利用率

资源类型	使用量	可用量	占用率
LUT (查找表)	1 226	303 600	0.40%
FF (触发器)	1 377	607 200	0.23%
IO (引脚)	168	600	28.00%

LUT 用于组合逻辑（译码、ALU、转发选择、分支判断等），FF 对应流水线寄存器和通用寄存器堆（ 32×32 bit = 1024 个触发器）。IO 占比相对较高（28%）是因为处理器顶层直接暴露了指令/数据总线的所有位宽端口。

5.4 关键路径分析

最差建立时间路径如图 4 所示，详情见表 14。

Summary	
Name	Path 1
Slack	0.446ns
Source	mem_wb_rd_reg[2]/C (rising edge-triggered cell FDCE clocked by sys_clk {rise@0.000ns fall@3.000ns period=6.000ns})
Destination	ex_mem_alu_result_reg[17]/D (rising edge-triggered cell FDCE clocked by sys_clk {rise@0.000ns fall@3.000ns period=6.000ns})
Path Group	sys_clk
Path Type	Setup (Max at Slow Process Corner)
Requirement	6.000ns (sys_clk rise@6.000ns - sys_clk rise@0.000ns)
Data Path Delay	5.528ns (logic 1.090ns (19.718%) route 4.438ns (80.282%))
Logic Levels	9 (LUT4=2 LUT5=1 LUT6=5 MUXF7=1)
Clock Path Skew	-0.025ns
Clock Uncertainty	0.035ns

图 4: Vivado 最差关键路径截图 (WNS = +0.446 ns)

表 14: 最差关键路径详情

属性	值
路径类型	Setup（建立时间，Slow Corner）
松弛（Slack）	+0.446 ns（满足）
时钟约束（Requirement）	6.000 ns
数据路径总延迟	5.528 ns
逻辑延迟（Logic）	1.090 ns（19.7%）
布线延迟（Route）	4.438 ns（80.3%）
逻辑级数	9（LUT4×2，LUT5×1，LUT6×5，MUXF7×1）
时钟路径偏斜	−0.025 ns
时钟不确定性	0.035 ns
起始节点（Source）	mem_wb_rd_reg[2]/C（MEM/WB 寄存器，目标寄存器地址位 2）
终止节点（Dest）	ex_mem_alu_result_reg[17]/D（EX/MEM 寄存器，ALU 结果位 17）

5.5 关键路径根因分析

关键路径从 MEM/WB 流水线寄存器的目标寄存器地址字段(mem_wb_rd_reg[2]) 出发，经过 9 级组合逻辑，落点于 EX/MEM 流水线寄存器的 ALU 结果字段：

(`ex_mem_alu_result_reg[17]`)。路径形成机制如下：

1. **转发比较逻辑**：EX 阶段数据转发检测需将 `mem_wb_rd` 与 `id_ex_rs1/id_ex_rs2` 进行相等比较，产生转发使能信号（约 2–3 级 LUT）；
2. **转发多路选择器**：根据比较结果，在 `wb_data`、`ex_mem_alu_result` 和 `id_ex_rdata1` 三路之间选择 EX 操作数（MUXF7 + 若干 LUT6）；
3. **ALU 运算**：选定的操作数经过 ALU 组合逻辑（加法器、移位等）计算最终结果，多位进位链进一步增加了级数；
4. **写入 EX/MEM 寄存器**：ALU 结果在时钟上升沿锁存到 `ex_mem_alu_result_reg`。

本质原因是：距离 2 拍的 MEM/WB→EX 转发路径串联了「rd 比较 → 多路选择 → ALU 计算」三段逻辑，共产生约 5.5 ns 的组合延迟。布线延迟占比 80.3% 说明 MEM/WB 寄存器与 EX 阶段逻辑在芯片上物理距离较远，若通过 Pblock 约束将两者归约至相邻区域，布线延迟可显著压缩，进一步提高 $F_{\max}$ 。

5.6 优化建议

表 15: 时序优化方案对比

方案	具体做法	预期效果	代价
提高时钟频率	将约束从 6 ns 收紧至 5.6 ns (≈ 180 MHz)	仍无违例（余量 0.446 ns）	近乎无代价
Pblock 布局约束	约束 EX 阶段与 MEM/WB 寄存器物理相邻	布线延迟从 4.4 ns 降至约 2 ns	需掌握 Vivado 布局约束语法
流水线 WB 寄存前移	在 MEM/WB 与 EX 之间增加一级寄存，使转发数据提前一拍稳定	彻底断开长链路	增加 1 拍流水延迟，写回更晚
减少转发优先级级联	将三路转发合并为优先级编码，减少多路选择器深度	减少约 1–2 级 LUT	需修改转发逻辑结构

当前设计已满足 6 ns 约束，最简单的下一步优化是将约束收紧至 5.6 ns (180 MHz)，利用现有 0.446 ns 余量验证是否仍无违例，无需修改任何 RTL 代码。

6 选做二：RV32I 完整指令集补全

6.1 补全目标

基础实验仅实现 13 条指令。本节在单周期 (single/) 与流水线 (pipeline/) 两种处理器上，补全了 RV32I 整数基础指令集中的剩余主要指令，新增 21 条，涵盖比较、算术移位、字节/半字访存、全部分支、LUI/AUIPC 和 JALR，详见[表 16](#)。

表 16: 新增 RV32I 指令一览

类别	指令	格式	功能描述	ALU 操作码
比较	SLT	R	$rd = (rs1 <_s rs2) ? 1 : 0$ (有符号)	4'd7
	SLTU	R	$rd = (rs1 <_u rs2) ? 1 : 0$ (无符号)	4'd8
	SLTI	I	$rd = (rs1 <_s imm) ? 1 : 0$	4'd7
	SLTIU	I	$rd = (rs1 <_u imm) ? 1 : 0$	4'd8
移位	SRA	R	$rd = rs1 \gg_a rs2[4 : 0]$ (算术右移)	4'd9
	SRAI	I	$rd = rs1 \gg_a imm[4 : 0]$	4'd9
Load	LB	I	$rd = \text{sign_ext}(\text{Mem}[\text{addr}][7 : 0])$	4'd0
	LH	I	$rd = \text{sign_ext}(\text{Mem}[\text{addr}][15 : 0])$	4'd0
	LBU	I	$rd = \text{zero_ext}(\text{Mem}[\text{addr}][7 : 0])$	4'd0
	LHU	I	$rd = \text{zero_ext}(\text{Mem}[\text{addr}][15 : 0])$	4'd0
Store	SB	S	$\text{Mem}[\text{addr}][7 : 0] = rs2[7 : 0]$	4'd0
	SH	S	$\text{Mem}[\text{addr}][15 : 0] = rs2[15 : 0]$	4'd0
分支	BNE	B	if $(rs1 \neq rs2)$ $PC += imm$	4'd1
	BGE	B	if $(rs1 \geq_s rs2)$ $PC += imm$ (有符号)	4'd7
	BLTU	B	if $(rs1 <_u rs2)$ $PC += imm$ (无符号)	4'd8
	BGEU	B	if $(rs1 \geq_u rs2)$ $PC += imm$	4'd8
立即数	LUI	U	$rd = imm_u$ (高 20 位)	4'd0
	AUIPC	U	$rd = PC + imm_u$	4'd0
跳转	JALR	I	$rd = PC + 4; PC = (rs1 + imm) \& \sim 1$	4'd0

6.2 各模块改动要点

6.2.1 ALU 新增运算 (ex.v, 单周期与流水线相同)

在 case(alu_op) 中新增三个操作码:

```

1 4'd7: alu_result = ($signed(src1) < $signed(src2)) ? 32'd1 : 32'
    d0; // SLT
2 4'd8: alu_result = (src1 < src2) ? 32'd1 : 32'd0;
    // SLTU
3 4'd9: alu_result = $signed(src1) >>> src2[4:0];
    // SRA

```

Code Listing 9: ALU 新增运算 (SLT/SLTU/SRA)

有符号比较使用 \$signed() 强制转换, 无符号比较直接用 < (Verilog 默认无符号语义)。

6.2.2 ID 译码器扩展 (id.v)

新增输出信号

- **单周期:** 新增 load_func3[2:0] 和 store_func3[2:0], 分别传递 func3 字段至 MEM 阶段, 用于区分字节/半字/字访存。
- **流水线:** 在上述基础上, 还新增 branch_type[2:0] (扩展自 2 位)、jalr、write_pc4, 将分支解析移至 EX 阶段。

LUI 与 AUIPC LUI 需将高 20 位立即数 imm_u 直接载入目标寄存器。实现方案: 强制 rs1=5'b0, 使寄存器堆读出 0, 再令 ALU 计算 $0 + \text{imm_u} = \text{imm_u}$ 。

```

1 7'b0110111: begin // LUI
2     rs1=5'b0; rs2=5'b0; reg_we=1; src1_sel=0; src2_sel=2'd1;
3     alu_op=4'd0; imm=imm_u; // ALU: 0 + imm_u = imm_u
4 end
5 7'b0010111: begin // AUIPC
6     rs1=5'b0; rs2=5'b0; reg_we=1; src1_sel=1; src2_sel=2'd1;
7     alu_op=4'd0; imm=imm_u; // ALU: PC + imm_u
8 end

```

Code Listing 10: LUI/AUIPC 译码 (以流水线 id.v 为例)

条件分支扩展 BLT/BGE 使用 SLT（操作码 7）而非减法来判断有符号大小关系，可避免有符号溢出（如 $\text{INT_MIN} - 1$ 会上溢导致减法符号位错误）。BLTU/BGEU 类似，使用 SLTU（操作码 8）。流水线版 `branch_type` 编码如表 17 所示。

表 17: 流水线 `branch_type` 编码与对应 ALU 操作

<code>branch_type</code>	指令	跳转条件	ALU 操作	跳转判定
3'd0	—	无分支	—	—
3'd1	BEQ	$\text{rs1} = \text{rs2}$	SUB (4'd1)	<code>alu_result = 0</code>
3'd2	BNE	$\text{rs1} \neq \text{rs2}$	SUB (4'd1)	<code>alu_result \neq 0</code>
3'd3	BLT	$\text{rs1} <_s \text{rs2}$	SLT (4'd7)	<code>alu_result = 1</code>
3'd4	BGE	$\text{rs1} \geq_s \text{rs2}$	SLT (4'd7)	<code>alu_result = 0</code>
3'd5	BLTU	$\text{rs1} <_u \text{rs2}$	SLTU (4'd8)	<code>alu_result = 1</code>
3'd6	BGEU	$\text{rs1} \geq_u \text{rs2}$	SLTU (4'd8)	<code>alu_result = 0</code>

JALR JALR 的跳转目标 ($\text{rs1} + \text{imm}$) 在 EX 阶段才能确定（需要转发后的 `rs1`）。两种处理器实现策略不同：

- **单周期**：ID 阶段直接读 `rdata1`（无转发问题），计算 `next_pc=(rdata1+imm_i)&~1`，同时令 ALU 计算 `PC+4` 作为返回地址写入 `rd`。
- **流水线**：设置 `jalr=1` 传至 ID/EX，令 ALU 计算 `rs1+imm`（目标地址），EX 阶段发现 `id_ex_jalr=1` 即触发跳转；写回地址通过 `write_pc4=1` 标志传递，WB 写回 `id_ex_pc+4`。

6.2.3 字节/半字访存（`mem.v`，单周期与流水线相同）

数据存储器 `data_mem.v` 为字节寻址（`addr` 直接作为字节地址），且提供独立的字节写使能 `we[3:0]`。`mem_stage` 根据 `store_func3` 和 `load_func3` 生成正确的字节使能与数据提取逻辑：

```

1 if (mem_we == 4'b1111 && mem_ce) begin
2     case (store_func3[1:0])
3         2'b00: mem_we_o = 4'b0001;    // SB: 仅最低字节
4         2'b01: mem_we_o = 4'b0011;    // SH: 低两字节
5         default: mem_we_o = 4'b1111; // SW: 全字
6     endcase
7 end

```

Code Listing 11: 字节写使能生成 (STORE)

```

1 case (load_func3)
2     3'b000: load_data = {{24{rdata[7]}}}, rdata[7:0]]; // LB:
        符号扩展字节
3     3'b001: load_data = {{16{rdata[15]}}}, rdata[15:0]]; // LH:
        符号扩展半字
4     3'b010: load_data = rdata; // LW
        : 全字
5     3'b100: load_data = {24'b0, rdata[7:0]]; // LBU
        : 零扩展字节
6     3'b101: load_data = {16'b0, rdata[15:0]]; // LHU
        : 零扩展半字
7 endcase

```

Code Listing 12: 加载数据提取 (LOAD)

6.2.4 流水线顶层重构 (pipeline/riscv.v)

流水线顶层需要针对新增指令进行较大改动，主要包括：

新增流水线寄存器字段 ID/EX 新增: id_ex_branch_type[2:0]、id_ex_jalr、id_ex_write_pc4、id_ex_load_func3[2:0]、id_ex_store_func3[2:0]。

EX/MEM 新增: ex_mem_load_func3、ex_mem_store_func3、ex_mem_write_pc4、ex_mem_return_pc (JALR 返回地址)。

MEM/WB 新增: mem_wb_write_pc4、mem_wb_return_pc。

EX 阶段分支/JALR 解析 条件分支和 JALR 的跳转均在 EX 阶段判断 (此时转发已稳定)，产生 ex_branch_taken 和 ex_branch_target，并冲刷 IF/ID 和 ID/EX 两级 (2 拍损失)：

```

1 assign ex_branch_taken =
2     (id_ex_branch_type==3'd1 && alu_result==32'd0) || // BEQ
3     (id_ex_branch_type==3'd2 && alu_result!=32'd0) || // BNE
4     (id_ex_branch_type==3'd3 && alu_result==32'd1) || // BLT
5     (id_ex_branch_type==3'd4 && alu_result==32'd0) || // BGE
6     (id_ex_branch_type==3'd5 && alu_result==32'd1) || // BLTU
7     (id_ex_branch_type==3'd6 && alu_result==32'd0) || // BGEU
8     id_ex_jalr;
9
10 assign ex_branch_target = id_ex_jalr ?
11     (alu_result & ~32'h1) : // JALR: (rs1+imm) & ~1

```

```
12 (id_ex_pc + id_ex_imm); // 条件分支: PC + imm_b
```

Code Listing 13: EX 阶段分支解析与 JALR 跳转

JALR 写回 (write_pc4 传递链) JALR 写回的是 PC+4 而非 ALU 结果, 通过 write_pc4 标志沿流水线传递, 在 WB 选择写回数据时最高优先级选择 return_pc:

```
1 assign wb_data = mem_wb_write_pc4 ? mem_wb_return_pc :
2           (mem_wb_mem_we == 4'b0 && mem_wb_mem_ce)
3           ? mem_wb_load_data : mem_wb_alu_result ;
```

Code Listing 14: WB 写回数据选择 (含 JALR)

6.3 测试程序设计与验证

6.3.1 测试策略

新编写 40 条指令的汇编测试程序 (机器码写入 machinecode.txt), 按序覆盖全部新增指令类型, 最终将通过次数计数器 x9 通过 SW x9, 12(x0) 写入 verify 地址 (字节地址 12), 预期值为 6。

6.3.2 测试程序流程

1. **算术/比较/移位** (指令 0-7): 验证 SLT、SLTU、SRA、SLTI、SLTIU、LUI。
 $x1 = -1, x2 = 1: \text{SLT} \Rightarrow x3 = 1; \text{SLTU} \Rightarrow x4 = 0; \text{SRA} \Rightarrow x5 = -1;$
 $\text{SLTI } x6, x1, 0 \Rightarrow x6 = 1 \ (-1 < 0); \text{SLTIU } x7, x0, 1 \Rightarrow x7 = 1 \ (0 < 1);$
 $\text{LUI } x8, 0xDEADB \Rightarrow x8 = 0xDEADB000。$
2. **分支测试** (指令 8-23): 依次测试 BNE、BLT、BGE、BLTU、BGEU, 每条分支均设计为应跳转, 跳过 1 条 NOP, 落入 ADDI x9, x9, 1。全部跳转正确则 x9 = 5。
3. **字节/半字访存** (指令 24-31):
 SB + LB (预期 x11 = 0xFFFFFFFF, 符号扩展 0xFF);
 LBU (预期 x12 = 0x000000FF, 零扩展);
 SH + LH (预期 x14 = 0xFFFFFFFF);
 LHU (预期 x15 = 0x0000FFFF)。
4. **AUIPC + JALR** (指令 32-37):
 AUIPC x16, 0 $\Rightarrow x16 = 128$ (当前 PC);
 JALR x17, x16, 20 $\Rightarrow$ 跳转至 $128 + 20 = 148$, 跳过 3 条 NOP,
 落入 ADDI x9, x9, 1 $\Rightarrow x9 = 6。$

5. **写入验证值** (指令 38): `SW x9, 12(x0) ⇒ verify = 6`。

6. **无限循环** (指令 39): `JAL x0, 0`, 防止 PC 越界。

6.3.3 验证结果

仿真输出:

```
--- result is 6 ---
```

单周期与流水线处理器均输出 `verify = 6`, 与预期一致, 证明新增 21 条指令的功能均正确。

7 选做三：自由拓展——AI 辅助开发

7.1 概述

本次实验的选做内容 (RV32I 指令集补全) 采用了 AI 辅助开发的工作流程, 借助 Claude AI 大语言模型协助进行架构设计、代码生成与验证, 最终显著提升了设计迭代效率。

7.2 AI 辅助内容

7.2.1 架构设计决策

在扩展流水线支持全 RV32I 时, 存在多个关键设计抉择, 由 AI 协助分析权衡:

- **条件分支在哪个阶段解析**: 单周期可在 ID 阶段直接使用 `rdata1/rdata2` 比较, 但流水线若在 ID 解析, 遇到数据相关时需等到转发完成, 反而增加阻塞。AI 分析后建议: **流水线分支移至 EX 阶段**——利用 EX 阶段已完成的转发值进行比较, 代价仅多冲刷 1 拍 (共 2 拍), 逻辑更清晰。
- **BLT/BGE 使用减法还是 SLT 判断**: 使用减法 (`rs1-rs2`) 检查符号位会在 `rs1 > 0, rs2 < 0` 时因有符号溢出产生错误结果。AI 指出应直接使用 SLT ALU 操作 (操作码 7), 其内部用 `$signed` 比较, 避免溢出陷阱。
- **JALR 返回地址路径**: ALU 被占用于计算目标地址 (`rs1+imm`), 返回地址 `PC+4` 需要单独传递。AI 建议引入 `write_pc4` 标志沿流水线传递, WB 阶段优先写回 `id_ex_pc+4`, 与 LOAD 路径并行存在, 避免额外 ALU 占用。
- **距离 3 拍 RAW 与写优先**: 在以 250 MHz (4 ns) 时钟进行的初步综合中, Vivado 报告了 `WNS = -2.301 ns` 的建立时间违例, 关键路径为「寄存器堆读 → ID 阶段组合 → 写优先选择 → 寄存器堆写」长链 (9 级 LUT, 近 6 ns)。AI 定位到根本

原因是 `register.v` 中的写优先 (Write-First) 组合逻辑, 建议将其替换为 ID/EX 捕获转发 (在 ID/EX 寄存器锁存时直接注入 `wb_data`), 以断开跨 WB→ID→写的组合链。优化后以 6 ns (167 MHz) 约束重新综合, 设计全部满足时序 (WNS = +0.446 ns, 零违例), 关键路径缩短为 MEM/WB 转发 → EX ALU 路径 (5.528 ns)。

7.2.2 代码生成与审查

AI 生成了以下模块的完整 Verilog 代码并逐行说明设计意图:

- `ex.v` (单周期 + 流水线): 新增 SLT/SLTU/SRA 的 ALU case 分支;
- `id.v` (单周期): 全 RV32I 组合译码逻辑, 含 LUI/AUIPC/JALR 特殊操作数处理;
- `id.v` (流水线): 在单周期版基础上抽取分支类型编码, 新增 `branch_type[2:0]`、`jalr`、`write_pc4` 输出;
- `mem.v` (单周期 + 流水线): 字节/半字 STORE 使能生成与 LOAD 数据提取;
- `riscv.v` (流水线顶层): 新增流水线寄存器字段、EX 阶段分支解析、双级冲刷逻辑、JALR 写回链路。

7.2.3 测试程序生成

AI 根据指令格式规范手工编码 40 条 RV32I 指令 (表 18), 涵盖所有新增指令类型, 并以追踪变量 `x9` 的方式设计可自动验证的测试策略 (最终 `verify=6` 表示全部 5 个分支均正确跳转, JALR 也正确执行)。

表 18: 测试程序机器码节选

机器码	汇编指令	预期结果	测试项
FFF00093	addi x1, x0, -1	x1=0xFFFFFFFF	初始化
0020A1B3	slt x3, x1, x2	x3=1	SLT
0020B233	sltu x4, x1, x2	x4=0	SLTU
4020D2B3	sra x5, x1, x2	x5=0xFFFFFFFF	SRA
DEADB437	lui x8, 0xDEADB	x8=0xDEADB000	LUI
00209463	bne x1, x2, +8	跳转, x9+1	BNE
0020C463	blt x1, x2, +8	跳转, x9+1	BLT
00115463	bge x2, x1, +8	跳转, x9+1	BGE
00106463	bltu x0, x1, +8	跳转, x9+1	BLTU
0000F463	bgeu x1, x0, +8	跳转, x9+1	BGEU
00A00023	sb x10, 0(x0)	Mem[0]=0xFF	SB
00000583	lb x11, 0(x0)	x11=0xFFFFFFFF	LB
00004603	lbu x12, 0(x0)	x12=0xFF	LBU
00000817	auipc x16, 0	x16=PC=128	AUIPC
01480867	jalr x17,x16,20	跳转至 148	JALR
00902623	sw x9, 12(x0)	verify=6	验证写回

7.3 AI 辅助开发体会

- **缩短迭代周期：**相比纯手工设计，AI 能快速给出详细的比特级编码计算和信号路径追踪，避免了在规范文档中反复查找位域定义的时间损耗，单个模块的设计-验证周期从数小时压缩至十几分钟。
- **问题诊断辅助：**当流水线 BEQ/BLT 在仿真中始终不跳转时，AI 通过分析原始 `riscv.v` 源代码，快速定位到 `branch_type` 信号在顶层虽已声明但从未被连接至 EX 阶段逻辑的根本原因，节省了大量调试时间。
- **局限性：**AI 不能替代真实的仿真工具，复杂冲突场景（如 Load-Use + 分支同时发生）仍需通过 QuestaSim 波形进行最终验证。AI 生成的代码也需要人工审查，确保符合实际硬件语义（如非阻塞赋值的使用、always 块敏感列表等）。
- **合规性说明：**本实验所有 AI 辅助均在实验规范允许的范围内进行（实验说明书明确将 AI 辅助开发列为自由拓展选项之一），所有生成代码均经本人理解、审查和修改后方才采用。

8 总结

8.1 设计总结

本实验在基础实验的 13 条指令基础上，通过选做内容进一步补全了完整 RV32I 指令集，主要完成内容如下：

1. **单周期处理器**：建立 IF-ID-EX-MEM-WB 五阶段组合逻辑数据通路，在 ID 阶段完成分支条件计算与 PC 控制，实现了 RV32I 主要指令的零缺陷执行。代码备份于 single/ 目录。
2. **流水线改造**：在单周期基础上于级间插入 4 组流水线寄存器，构建五级流水线结构；实现三条数据转发路径（EX/MEM→EX、MEM/WB→EX、ID/EX 捕获转发）解决 RAW 冲突；Load-Use 冲突通过 stall_req 机制暂停 1 拍处理；控制冲突通过冲刷 IF/ID 和 ID/EX 两级处理（JAL 冲刷 1 拍，条件分支/JALR 冲刷 2 拍）。
3. **Vivado 综合分析（选做）**：通过 Vivado 综合分析得到了综合后时序、资源利用率报告，根据报告，综合布线性能已经很高，难以优化。
4. **RV32I 指令集补全（选做）**：在两种处理器上新增 21 条指令，涵盖 SLT/SLTU/SRA、字节/半字访存（LB/LH/LBU/LHU/SB/SH）、全部分支（BNE/BGE/BLTU/BGEU）、LUI/AUIPC/JALR；编写了 40 条指令的自动化验证测试程序，仿真验证两种处理器均输出预期结果 verify = 6。
5. **AI 辅助开发（选做）**：借助 Claude AI 大语言模型协助架构决策、代码生成与测试程序编码，并在实验报告中如实记录了 AI 辅助的范围与局限性。

8.2 性能分析

表 19: 单周期与五级流水线处理器性能对比

指标	单周期	五级流水线
时钟周期约束	最长路径（含 LW 全通路）	最长单阶段路径（约 1/5）
理论最高频率	低（100 MHz 量级）	高（约 5×, 500 MHz 量级）
理想 CPI	1	趋近 1
Load-Use 代价	无（固定 1 周期/指令）	+1 周期/发生
Branch 代价	无	+1 周期/发生
实际吞吐量	低	高（频率提升远大于冲突损失）
设计复杂度	低	高（需处理三类冲突）

8.3 心得体会

通过本次实验，对流水线处理器设计的核心难点有了切身体会：

- **数据相关无处不在**：从单周期改为流水线的第一步便要系统梳理所有可能的写后读场景，距离 1/2/3 拍的处理策略各有差异，不可遗漏。
- **Load-Use 的特殊性**：LW 的结果在 MEM 阶段末才就绪，是流水线中唯一无法靠纯转发消除的 RAW 冲突，必须引入暂停机制，这也是通常 LW 后需要编译器主动插入独立指令以隐藏延迟的原因。
- **控制信号也需打拍**：流水线寄存器不仅要传递数据（rdata1/2、imm、PC），还需携带与数据对齐的控制信号（reg_we、mem_ce、mem_we 等），确保在正确的阶段发出正确的控制。
- **JAL 的操作数特殊性**：JAL 需要以 PC 作为 src1，而 PC 不在寄存器堆中，必须在 ID/EX 寄存器中单独传递 PC 并引入 src1_sel 信号，这是将单周期直接复用为流水线子模块时容易忽略的细节。